

Hadoop与Spark

Hive



Hive简介、部署与开发

◆ Hive简介

- ◆ Hive是基于Hadoop的一个数据仓库系统
- ◆ 它将结构化的数据文件映射为一张数据库表
- ◆ Hive把Hive查询语言(Hive Query Language, 简称HQL, 类似SQL的查询语言)所编写的语句, 转换为MapReduce作业(Job), 然后在Hadoop平台上运行
- ◆ 使用Hive, 用户通过类似SQL的查询语言, 快速实现简单的数据分析和统计, 学习成本低
- ◆ 大量熟悉SQL语言的开发者、数据分析人员, 都可以很快地学会使用Hive进行数据分析, 不必开发专门的MapReduce应用

Hive简介、部署与开发

◆ Hive简介: Hive与传统的关系数据库的差别

- ◆ (1) Hive使用Hadoop的HDFS分布式文件系统存储数据库数据, 而关系数据库则是使用服务器的本地文件系统存储数据库数据。
- ◆ (2) Hive使用MapReduce计算模型实现查询处理, 关系数据库则是使用自有的查询处理模型, 比如Shared Nothing无共享架构下的并行查询处理模型。
- ◆ (3) 关系数据库主要为高性能查询而设计, Hive的设计目标则是对海量数据进行分析处理, 其实时性没有关系数据库好, 因此Hive和关系数据库的应用场景是不同的。
- ◆ (4) 基于Hadoop平台, Hive具有强大的扩展能力, 可以很容易地对存储能力和计算能力进行扩展。而关系数据库在扩展性方面则要差很多, 目前没有关系数据库系统能够扩展到1,000台以上的集群规模。
- ◆ (5) 关系数据库在数据加载的时候, 要求数据必须符合数据库表的模式(表结构), 否则不能加载, 称为“写时模式”(Writing Time Schema)。Hive在数据加载的时候, 无需进行模式检查, 在读取数据的时候, 再对数据以一定的模式进行解释, 称为“读时模式”(Reading Time Schema)
- ◆ (6) 关系数据库, 可以对某行数据进行更新、删除等操作, 并且支持事务处理。Hive不支持某个具体数据行的修改, 它只支持数据的追加功能。Hive也不支持事务和索引。Hive的设计目标是对海量数据进行分析处理, 一般需要对数据进行全部扫描。对于更新操作, 需要将原来表格里数据进行更改(转换)后, 写入一个新的表格

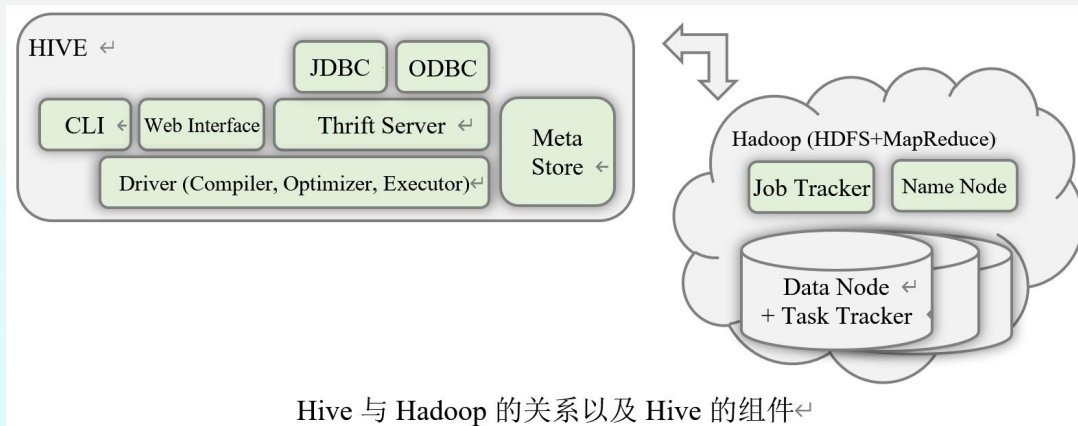
Hive简介、部署与开发

- ◆ Hive简介

- ◆ Hive的组件

- ◆ Hive系统，包括如下组件：命令行接口CLI (Command Line Interface)、JDBC/ODBC驱动程序、Thrift Server服务器、WEB 图形用户界面(Graphic User Interface, GUI)、元信息管理器MetaStore、和Driver(包括Complier、Optimizer和Executor)等

- ◆ 这些组件可以分为两大类，即服务端组件和客户端组件



Hive 与 Hadoop 的关系以及 Hive 的组件

Hive简介、部署与开发

- ◆ Hive的组件
 - ◆ 服务器端组件
 - ◆ Hive的服务端组件包括：
 - ◆ (1) Driver: 该组件包括编译器(Compiler)、优化器(Optimizer)、和执行器(Executor), 它的作用是将用户编写的HQL查询语句进行解析、编译、优化, 生成执行计划, 然后调用底层的MapReduce计算模型来执行
 - ◆ (2) 元信息管理器MetaStore: 是元数据管理组件, 负责存储Hive的元数据。Hive的元数据存储在关系数据库里, Hive支持的关系数据库有Derby、MySQL、Oracle等。元数据对于Hive的正确运行, 举足轻重。Hive允许把MetaStore服务独立出来, 安装到远程的服务器集群里, 从而解耦Hive服务和MetaStore服务, 保证Hive运行的健壮性
 - ◆ (3) Thrift Server服务器: Thrift Server服务器是Facebook开发的一个软件框架, 用于开发可扩展的、跨语言的服务接口。Hive集成了Thrift Server服务器, 支持不同的编程语言调用Hive的接口

Hive简介、部署与开发

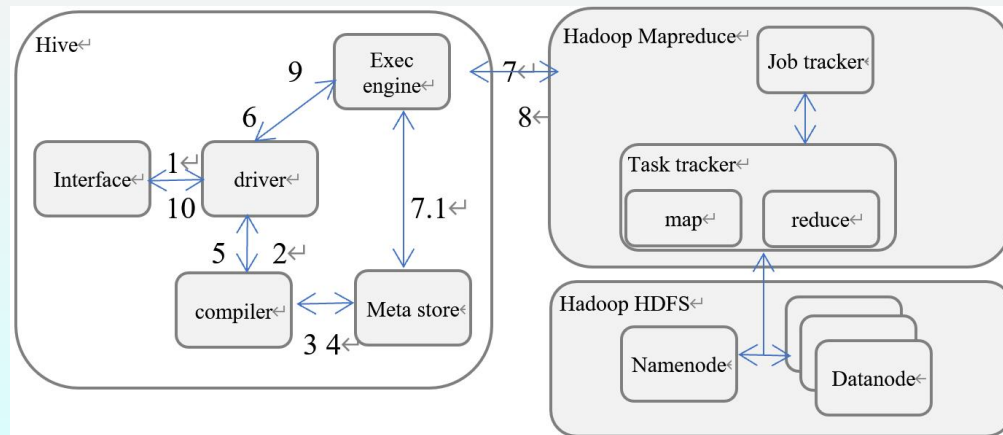
- ◆ Hive的组件
 - ◆ 客户端组件
 - ◆ Hive的客户端组件包括：
 - ◆ (1) 命令行接口CLI: 命令行接口允许用户交互式地使用Hive
 - ◆ (2) Thrift客户端: 包括JDBC和ODBC驱动程序
 - ◆ (3) WEB图形用户界面(GUI): Hive客户端提供了一种通过网页, 访问Hive服务的方式
 - ◆ 通过WEB图形用户界面访问Hive, 必须首先启动HWI(Hive Web Interface)服务

Hive简介、部署与开发

◆ Hive简介

◆ Hive查询处理过程

- ◆ 用户提交SQL查询给Driver，Driver把查询交给Compiler进行编译和检查，中间需要参考元信息Metastore
- ◆ SQL查询转换成MapReduce Job以后，交给Execution Engine执行，调用Hadoop平台的MapReduce运行时(RunTime)执行MapReduce Job，MapReduce Job存取HDFS，对数据进行处理。查询结果返回给Driver，并最后返回用户



Hive 查询处理过程

Hive简介、部署与开发

◆ Hive数据模型

- ◆ Hive支持的数据类型，包括整型(Integer)、浮点型(Float)、双精度浮点型(Double)，以及字符串(String)等。Hive还支持更加复杂的数据类型，包括映射(Map)、列表(List)和结构(Struct)
- ◆ 这些复杂类型可以通过嵌套，表达更加复杂的类型。除此之外，Hive允许用户自己定义类型(Type)和函数(Function)，来扩展系统
- ◆ HQL和标准的SQL语言非常相似，熟悉SQL语言的开发者和数据分析者，很容易理解和掌握

Hive简介、部署与开发

◆ Hive数据模型

- ◆ Hive使用传统数据库使用的表格(Table)、行(Row)、列(Column)、分区(Partition)等概念，易于理解。Hive的数据模型包括几个主要的管理层次，分别是数据库(Database)、表格(Table)、分区(Partition)、桶(Bucket)等。
- ◆ (1) 数据库(Database): 相当于关系数据库里的命名空间(Namespace)。它的作用是将不同用户的数据，隔离到不同的数据库和模式中
- ◆ (2) 表格(Table): Hive的表格，逻辑上由数据和描述数据格式的元数据组成。表格的数据存放在分布式文件系统里，即HDFS，元数据存储在关系数据库里。当我们创建一张Hive表格，还没有为表格加载数据的时候，该表格在分布式文件系统(即HDFS)上，就是一个目录。
- ◆ Hive里的表格分为两种类型，一种叫托管表，这种表的数据文件存储在Hive的数据仓库里，一种叫外部表，这种表的数据文件可以存放在Hive数据仓库外部的分布式文件系统上，也可以放到Hive数据仓库里
- ◆ Hive的数据仓库就是HDFS上的一个目录，这个目录是Hive数据文件存储的默认路径，相关元信息会存放到元数据库里

Hive简介、部署与开发

◆ Hive数据模型

◆ **(3) 分区(Partition):** Hive里的分区是根据“分区列”的值，对表格的数据进行粗略划分的机制。在Hive存储上，表格的主目录下的一个子目录，它的名字就是我们定义的分区列的名字。分区列不是表里的某个实际字段(列)，而是一个虚拟的属性列。根据这个列，划分和存储表格里的数据。使用分区是为了加快查询速度而设计的，如果我们的查询，仅仅需要存取部分数据的时候，就没有必要进行全表扫描

◆ 比如，我们通过如下HQL语句建立logs(日志)表格，表格的字段包括ts(时间戳)和line(日志的内容)，并且进行分区，分区字段为dt(日期)和country(国家)

```
Create Table logs (ts bigint, line string) Partitioned By (dt string, country string);
```

◆ 我们可以在Hive命令行程里，执行如何语句把数据导入这张表

```
Load data local inpath '/home/hadoop/file01.txt' into table logs partition  
(dt='2013-06-02', country='cn');
```

Hive简介、部署与开发

- ◆ Hive数据模型

- ◆ (3) 分区(Partition)

- ◆ 那么在hive数据仓库里，数据实际存储的路径(HDFS分布式文件系统路径)如下所示。注意，我们还通过另外的命令，把dt=2013-06-02、country=us的数据导了进去。

```
/user/hive/warehouse/logs/dt=2013-06-02/country=cn/file1.txt  
/user/hive/warehouse/logs/dt=2013-06-02/country=cn/file2.txt  
/user/hive/warehouse/logs/dt=2013-06-02/country=us/file3.txt  
/user/hive/warehouse/logs/dt=2013-06-02/country=us/file4.txt  
...
```

- ◆ 我们看到在表格logs对应的目录下，有了两层子目录，比如dt=2013-06-02和country=cn。当我们执行如下SQL查询的时候，查询操作只会扫描file1.txt和file2.txt文件

```
Select ts, dt, line from logs where country='cn';
```

Hive简介、部署与开发

- ◆ Hive数据模型

- ◆ (4) 桶(Bucket): Table和Partition都是目录级别的数据拆分。使用桶的表格, 将数据文件按一定规律拆分成多个文件, 每个桶就是表目录(或者分区子目录)里的一个文件。数据的分桶, 一般通过Hash函数实现。创建表的时候, 用户指定需要的桶的数量, 以及使用哪个数据列进行分桶操作(Bucket the Data)
- ◆ 当用户执行一个采样(Sample)查询的时候, Hive可以使用分桶信息, 有效地裁剪数据(Prune Data)。比如有一个数据库表, 每个(子)目录下有32个桶。我们可以提取每个目录下的第一个桶文件, 生成一个1/32数据量的采样

pause



Hive简介、部署与开发

◆ Hive安装、配置和运行

- ◆ 安装Hive之前，首先安装JDK和Hadoop，并且停止所有的Hadoop服务，包括HDFS和YARN
- ◆ 因为Hive在进行查询处理时，只是做HQL到MapReduce的转换，并且把元数据存在MySQL/Derby上
- ◆ 所以只需要在主节点安装、配置一个Hive即可，适用于单节点部署，也适用于集群部署

Hive简介、部署与开发

- ◆ Hive安装、配置和运行
 - ◆ 使用MySQL进行元信息管理

安装MySQL。

`yum install mariadb mariadb-server` \\ 询问是否要安装，输入Y即可自动安装

拷贝MySQL缺省配置文件。

`cp /usr/share/mysql/my-huge.cnf /etc/my.cnf`

\\ 拷贝配置文件，如果/etc目录下面默认有一个my.cnf，直接覆盖即可

第一次运行MySQL，要初始化数据库。

`mysql_secure_installation`

\\ 设置user name root

\\ 设置password rootroot

Hive简介、部署与开发

- ◆ Hive安装、配置和运行
 - ◆ 使用MySQL进行元信息管理

运行MySQL。

systemctl start mariadb.service \\ 启动MariaDB

systemctl stop mariadb.service \\ 停止MariaDB

systemctl restart mariadb.service \\ 重启MariaDB

systemctl enable mariadb.service \\ 设置开机启动

通过mysql客户端程序连接MySQL。

mysql -uroot -prootroot

\\ 用客户端软件mysql连接mysql服务器

\\ 连接成功，表示MySQL已经安装好

Hive简介、部署与开发

- ◆ Hive安装、配置和运行
 - ◆ 使用MySQL进行元信息管理

对MySQL进行授权。

```
mysql -uroot -prootroot
```

\\ 运行mysql客户端，执行如下SQL命令

```
show databases;
```

```
CREATE USER 'root'@'192.168.31.129' IDENTIFIED BY 'rootroot';
```

```
GRANT ALL PRIVILEGES ON metastore_db.* TO 'root'@'192.168.31.129' WITH  
GRANT OPTION;
```

\\ 或者授予更大权限

```
GRANT ALL PRIVILEGES ON *.* TO 'root'@'%' IDENTIFIED BY 'rootroot' WITH  
GRANT OPTION;
```

```
FLUSH PRIVILEGES;
```

Hive简介、部署与开发

- ◆ Hive安装、配置和运行
 - ◆ 使用MySQL进行元信息管理
 - ◆ 从<https://downloads.mariadb.org/connector-java/>下载mariadb-java-client-1.3.7.jar到/opt/linuxsir/hive/lib目录下

```
cp /opt/linuxsir/mariadb-java-client-1.3.7.jar /opt/linuxsir/hive/lib  
ls /opt/linuxsir/hive/lib/mariadb-java-client-1.3.7.jar
```

Hive简介、部署与开发

- ◆ Hive安装、配置和运行
 - ◆ 使用MySQL进行元信息管理
 - ◆ 从<https://downloads.mariadb.org/connector-java/>下载mariadb-java-client-1.3.7.jar到/opt/linuxsir/hive/lib目录下
 - ◆ 先下载Hive，解压缩安装

```
cp /opt/linuxsir/mariadb-java-client-1.3.7.jar /opt/linuxsir/hive/lib  
ls /opt/linuxsir/hive/lib/mariadb-java-client-1.3.7.jar
```

Hive简介、部署与开发

- ◆ Hive安装、配置和运行
- ◆ 下载和安装Hive
- ◆ 从<http://www-eu.apache.org/dist/hive/stable-2/>下载hive, 上传到/opt/linuxsir, 然后解压并重命名为/opt/linuxsir/hive

```
cd /opt/linuxsir  
tar xzvf apache-hive-2.3.3-bin.tar.gz  
ls apache-hive-2.3.3-bin  
  
mv apache-hive-2.3.3-bin /opt/linuxsir/hive
```

Hive简介、部署与开发

- ◆ Hive安装、配置和运行
- ◆ 创建hive数据存储文件夹，修改文件夹权限

```
mkdir -p /opt/linuxsir/hive/data  
chmod a+rwX /opt/linuxsir/hive/data
```

Hive简介、部署与开发

- ◆ Hive安装、配置和运行
- ◆ 设置Hive环境变量

```
echo "" >> ~/.bashrc
echo "# Hive Variables" >> ~/.bashrc
echo "export HIVE_HOME=/opt/linuxsir/hive" >> ~/.bashrc
echo "export HIVE_CONF_DIR=\$HIVE_HOME/conf" >> ~/.bashrc
echo "export PATH=\$HIVE_HOME/bin:\$PATH" >> ~/.bashrc

echo "export CLASSPATH=\$CLASSPATH:/opt/linuxsir/hive/lib/*" >> ~/.bashrc

cat ~/.bashrc
source ~/.bashrc
```

Hive简介、部署与开发

- ◆ Hive安装、配置和运行
- ◆ 在HDFS上为Hive创建目录
- ◆ 需要启动HDFS和YARN

```
cd /opt/linuxsir/hadoop/bin
hdfs dfs -rmr /user/hive

hdfs dfs -mkdir -p /user/hive/warehouse
hdfs dfs -mkdir -p /user/hive/tmp
hdfs dfs -mkdir -p /user/hive/log

hdfs dfs -chmod a+rwX /user/hive/warehouse
hdfs dfs -chmod a+rwX /user/hive/tmp
hdfs dfs -chmod a+rwX /user/hive/log

hdfs dfs -ls /user/hive
```

Hive简介、部署与开发

- ◆ Hive安装、配置和运行
- ◆ 创建Hive的日志(logs)目录

```
mkdir /opt/linuxsir/hive/logs
```

Hive简介、部署与开发

- ◆ Hive安装、配置和运行
- ◆ 在/opt/linuxsir/hive/conf目录下，从模板拷贝并编辑hive-exec-log4j2.properties

```
cd /opt/linuxsir/hive/conf  
cp hive-exec-log4j2.properties.template hive-exec-log4j2.properties
```

- ◆ 修改内容如下

```
property.hive.log.dir=/opt/linuxsir/hive/logs
```

Hive简介、部署与开发

- ◆ Hive安装、配置和运行
- ◆ 在/opt/linuxsir/hive/conf目录下，从模板拷贝hive-log4j2.properties

```
cd /opt/linuxsir/hive/conf  
cp hive-log4j2.properties.template hive-log4j2.properties
```

Hive简介、部署与开发

- ◆ Hive安装、配置和运行
- ◆ 在/opt/linuxsir/hive/conf目录下，从模板拷贝并编辑hive-env.sh

```
cd /opt/linuxsir/hive/conf  
cp hive-env.sh.template hive-env.sh
```

- ◆ 修改内容如下

```
export JAVA_HOME=/opt/linuxsir/java/jdk  
export HADOOP_HOME=/opt/linuxsir/hadoop  
export HIVE_CONF_DIR=/opt/linuxsir/hive/conf  
export HIVE_AUX_JARS_PATH=/opt/linuxsir/hive/lib
```

Hive简介、部署与开发

- ◆ Hive安装、配置和运行
- ◆ 在/opt/linuxsir/hive/conf目录下，从模板拷贝并编辑hive-site.xml

```
cd /opt/linuxsir/hive/conf  
cp hive-default.xml.template hive-site.xml
```

- ◆ 做文件内容的局部修改，具体如下

```
<property>  
  <name>javax.jdo.option.ConnectionURL</name>  
  <value> jdbc:mysql://hd-master:3306/metastore_db?createDatabaseIfNotExist=true </value>  
</property>  
<property>  
  <name> javax.jdo.option.ConnectionDriverName </name>  
  <value> org.mariadb.jdbc.Driver </value>  
</property>  
<property>  
  <name> javax.jdo.option.ConnectionUserName </name>  
  <value>root</value>  
</property>  
<property>  
  <name> javax.jdo.option.ConnectionPassword </name>  
  <value>rootroot</value>  
</property>
```

Hive简介、部署与开发

- ◆ Hive安装、配置和运行
- ◆ 在/opt/linuxsir/hive/conf目录下，从模板拷贝并编辑hive-site.xml

```
cd /opt/linuxsir/hive/conf  
cp hive-default.xml.template hive-site.xml
```

- ◆ 做文件内容的局部修改，具体如下

```
<property>  
<name>hive.metastore.warehouse.dir</name>  
<value>/user/hive/warehouse</value>  
  <!-- <value>hdfs://hd-master:9000/user/hive/warehouse</value> -->  
</property>  
<property>  
<name>hive.exec.scratchdir</name>  
<value>/user/hive/tmp</value>  
  <!-- <value>hdfs://hd-master:9000/user/hive/tmp</value> -->  
</property>  
<property>  
<name>hive.querylog.location</name>  
<value>/user/hive/log</value>  
  <!-- <value>hdfs://hd-master:9000/user/hive/log</value> -->  
</property>
```

Hive简介、部署与开发

- ◆ Hive安装、配置和运行
- ◆ 在/opt/linuxsir/hive/conf/hive-site.xml文件里，进行hive.metastore.uris的配置
- ◆ hive.metastore.uris的端口设置缺省为9083，修改成19083

```
<property>  
<name> hive.metastore.uris </name>  
<value> thrift://192.168.31.129:19083</value>  
</property>
```

Hive简介、部署与开发

- ◆ Hive安装、配置和运行
- ◆ 在hive-site.xml文件里，查找`${system:java.io.tmpdir}`替换成指定文件夹，即/opt/linuxsir/hive/tmp

`${system:java.io.tmpdir}` 替换成/opt/linuxsir/hive/tmp

\\需要事先创建tmp目录，在终端上运行命令`mkdir /opt/linuxsir/hive/tmp`

Hive简介、部署与开发

- ◆ Hive安装、配置和运行
- ◆ 启动
- ◆ 启动Hive之前，必须先启动hdfs和yarn

```
cd /opt/linuxsir/hadoop/sbin  
./start-dfs.sh  
./start-yarn.sh
```

```
jps  
ssh root@192.168.31.140 jps  
ssh root@192.168.31.141 jps
```

Hive简介、部署与开发

- ◆ Hive安装、配置和运行
- ◆ 启动
 - ◆ 删除metastore_db



初始化mysql只做一次

```
mysql -uroot -prootroot  
\\ 启动MySQL后，把metastore_db删除  
drop database metastore_db;  
exit;
```

- ◆ 启动hive之前，初始化MySQL的metastore_db数据库

```
cd /opt/linuxsir/hive  
./bin/schematool -dbType mysql -initSchema
```

\\ 如果出错的话，对/etc/my.cnf配置文件进行改动，具体为
“binlog_format=mixed”

\\ 然后重启MySQL(参考6.3.1)，删除metastore_db数据库，再运行schematool

Hive简介、部署与开发

- ◆ Hive安装、配置和运行
- ◆ 启动
 - ◆ 新建数据文件

```
cd /opt/linuxsir
touch hive-test.txt

echo "1 hadoop" >> hive-test.txt
echo "2 hive" >>hive-test.txt
echo "3 hbase" >>hive-test.txt
echo "4 hello" >>hive-test.txt

cat hive-test.txt
```

Hive简介、部署与开发

- ◆ Hive安装、配置和运行
- ◆ 启动
 - ◆ 如果已经在/opt/linuxsir/hive/conf/hive-site.xml里面配置hive.metastore.uris, 那么必须首先启动metastore
 - ◆ 也就是首先运行metastore, 再运行hive shell

```
cd /opt/linuxsir/hive  
./bin/hive --service metastore -p 19083&
```

- ◆ 启动hive shell

```
cd $HIVE_HOME/bin  
./hive
```

Hive简介、部署与开发

- ◆ Hive安装、配置和运行
- ◆ 启动
 - ◆ 在Hive Shell里面，进行创建数据库表，进行增、删、改、查等操作

```
set hive.execution.engine=mr;
```

```
show databases;
```

```
create table test(a string, b int);
```

```
show tables;
```

```
desc test;
```

```
CREATE TABLE IF NOT EXISTS words (id INT,word STRING)
```

```
ROW FORMAT DELIMITED FIELDS TERMINATED BY " "
```

```
LINES TERMINATED BY "\n";
```

```
LOAD DATA LOCAL INPATH '/opt/linuxsir/hive-test.txt' OVERWRITE INTO TABLE words;
```

```
select * from words;
```

```
insert into words values(5,'nihao');
```

```
select * from words;
```

```
select * from words order by id;
```

```
select id, count(*) from words group by id order by id;
```

pause



Hive简介、部署与开发

- ◆ Hive安装、配置和运行
 - ◆ 问题1：提示“FAILED: Execution Error, return code -101 from org.apache.hadoop.hive ql.exec.mr.MapRedTask.”。
 - ◆ 查看hd-master下yarn-root-resourcemanager-hd-master.log, 有提示“Invalid resource request, requested memory < 0, or requested memory > max configured, requestedMemory=1280, maxMemory=512”

```
修改/opt/linuxsir/Hadoop/etc/hadoop/yarn-site.xml。  
<property>  
  <name>yarn.nodemanager.resource.memory-mb</name>  
  <value>2048</value>  
</property>  
  
<property>  
  <name>yarn.scheduler.maximum-allocation-mb</name>  
  <value>2048</value>  
</property>
```

Hive简介、部署与开发

- ◆ Hive安装、配置和运行
 - ◆ 问题1：提示“FAILED: Execution Error, return code -101 from org.apache.hadoop.hive ql.exec.mr.MapRedTask.”。
 - ◆ 查看hd-master下yarn-root-resourcemanager-hd-master.log, 有提示“Invalid resource request, requested memory < 0, or requested memory > max configured, requestedMemory=1280, maxMemory=512”

```
修改hdfs上/user/hive目录的权限。  
cd /opt/linuxsir/hadoop/bin  
./hdfs dfs -chmod a+rwX /user/hive
```

- ◆ 然后停止hive shell, 停止yarn, 停止hdfs
- ◆ 用scp命令把yarn-site.xml复制到hd-slave1和hd-slave2
- ◆ 然后再启动hdfs, 启动yarn, 启动hive shell, 再试一试

Hive简介、部署与开发

- ◆ Hive安装、配置和运行
 - ◆ 问题2: **hive cli**界面由于**hdfs in safemode**启动不起来
 - ◆ 解决办法如下。退出hive shell, 执行如下命令, 再启动hive shell试一试

```
cd /opt/linuxsir/hadoop/bin  
hdfs dfsadmin -safemode leave
```

Hive简介、部署与开发

◆ Hive安装、配置和运行

- ◆ 问题3: 提示“FAILED: Execution Error, return code -101 from org.apache.hadoop.hive ql.exec.mr.MapRedTask.org.apache.hadoop.mapreduce.v2.util.MRApps.addLog4jSystemProperties (Lorg/apache/hadoop/mapred/Task;Ljava/util/List;Lorg/apache/hadoop/conf/Configuration;)V”

在hive/conf/ hive-log4j.properties 文件中进行Hive日志配置, 具体如下。

hive.root.logger=WARN,DRFA

hive.log.dir=/tmp/\${user.name} # 默认的存储位置, 可以改成
/opt/linuxsir/hive/logs

hive.log.file=hive.log # 默认的文件名

出错以后查看/opt/linuxsir/hive/logs/hive.log文件, 了解出错信息

Hive简介、部署与开发

- ◆ Hive安装、配置和运行
 - ◆ 问题3：提示“FAILED: Execution Error, return code -101 from org.apache.hadoop.hive ql.exec.mr.MapRedTask.org.apache.hadoop.mapreduce.v2.util.MRApps.addLog4jSystemProperties (Lorg/apache/hadoop/mapred/Task;Ljava/util/List;Lorg/apache/hadoop/conf/Configuration;)V”

这个问题的解决办法如下。

```
cd /opt/linuxsir/hadoop/share/hadoop/mapreduce  
cp hadoop-mapreduce-client-*.jar /opt/linuxsir/hive/lib
```

Hive简介、部署与开发

- ◆ Hive安装、配置和运行

- ◆ 问题4: 访问<http://hd-master:50070/dfshealth.html#tab-overview>, 得到如下信息。

Diagnostics: org.apache.hadoop.hdfs.BlockMissingException
There are 6 missing blocks. The following files may be corrupted:

```
blk_1073742055 /user/root/.hiveJars/hive-exec-2.1.1-  
5f4a7e952d29bb8013edd30bbc39476ec56bc381b96b0530a6b2fbbf28e309  
d3.jar  
blk_1073745585 /apps/tez.tar.gz  
blk_1073756690 /tmp/hadoop-  
yarn/staging/root/.staging/job_1519795952327_0001/job.jar  
blk_1073756691 /tmp/hadoop-  
yarn/staging/root/.staging/job_1519795952327_0001/job.split  
blk_1073756692 /tmp/hadoop-  
yarn/staging/root/.staging/job_1519795952327_0001/job.splitmetainfo  
blk_1073756693 /tmp/hadoop-  
yarn/staging/root/.staging/job_1519795952327_0001/job.xml
```

Hive简介、部署与开发

◆ Hive安装、配置和运行

- ◆ 问题4: 访问<http://hd-master:50070/dfshealth.html#tab-overview>, 得到如下信息。

退出hive shell, 通过删除和重新创建Hive在HDFS的目录, 再启动hive shell试一试。

```
cd /opt/linuxsir/hadoop/bin \\删除目录
```

```
./hdfs dfs -rmr /tmp
```

```
cd /opt/linuxsir/hadoop/bin \\删除目录
```

```
./hdfs dfs -rmr /user/root
```

```
./hdfs dfs -rmr /user/hive/warehouse \\删除目录
```

```
./hdfs dfs -rmr /user/hive/tmp
```

```
./hdfs dfs -rmr /user/hive/log
```

```
hdfs dfs -mkdir -p /user/hive/warehouse \\创建目录
```

```
hdfs dfs -mkdir -p /user/hive/tmp
```

```
hdfs dfs -mkdir -p /user/hive/log
```

```
hdfs dfs -chmod a+rwX /user/hive/warehouse \\授权
```

```
hdfs dfs -chmod a+rwX /user/hive/tmp
```

```
hdfs dfs -chmod a+rwX /user/hive/log
```

```
./hdfs dfs -ls /user/hive
```

Hive简介、部署与开发

◆ Hive安装、配置和运行

- ◆ 问题4: 访问<http://hd-master:50070/dfshealth.html#tab-overview>, 得到如下信息。

如果HDFS分布式文件系统有问题, 可以参考第4章, 重新格式化HDFS

```
rm -rf /opt/linuxsir/hadoop/tmp
rm -rf /opt/linuxsir/hadoop/hdfs
```

```
mkdir /opt/linuxsir/hadoop/tmp      \\创建tmp目录
mkdir -p /opt/linuxsir/hadoop/hdfs/data /opt/linuxsir/hadoop/hdfs/name \\创建hdfs的data、name子目录
```

```
ssh root@192.168.31.130 rm -rf /opt/linuxsir/hadoop/tmp
ssh root@192.168.31.130 rm -rf /opt/linuxsir/hadoop/hdfs
```

```
ssh root@192.168.31.130 mkdir /opt/linuxsir/hadoop/tmp
ssh root@192.168.31.130 mkdir -p /opt/linuxsir/hadoop/hdfs/data /opt/linuxsir/hadoop/hdfs/name
```

```
ssh root@192.168.31.131 rm -rf /opt/linuxsir/hadoop/tmp
ssh root@192.168.31.131 rm -rf /opt/linuxsir/hadoop/hdfs
```

```
ssh root@192.168.31.131 mkdir /opt/linuxsir/hadoop/tmp
ssh root@192.168.31.131 mkdir -p /opt/linuxsir/hadoop/hdfs/data /opt/linuxsir/hadoop/hdfs/name
```

```
cd /opt/linuxsir/hadoop/bin
./hdfs namenode -format
```

pause



Hive简介、部署与开发

- ◆ Hive安装、配置和运行
 - ◆ Hiveserver2与beeline
 - ◆ 我们可以启动Hiveserver2(JDBC Server), 使用beeline (JDBC Client)连接Hiveserver2对数据进行操作
 - ◆ 启动Hiveserver2, 需要先启动hdfs/yarn

```
\\启动hdfs和yarn
cd /opt/linuxsir/hadoop/sbin
./start-dfs.sh
./start-yarn.sh

Jps \\查看进程是否已经正常启动
ssh root@192.168.31.130 jps
ssh root@192.168.31.131 jps
```

Hive简介、部署与开发

- ◆ Hive安装、配置和运行
 - ◆ Hiveserver2与beeline
 - ◆ 启动metastore

```
cd /opt/linuxsir/hive
\\启动metastore
./bin/hive --service metastore -p 19083&

\\ 等待metastore启动
netstat -ntlp|grep 19083
```

Hive简介、部署与开发

- ◆ Hive安装、配置和运行
 - ◆ Hiveserver2与beeline
 - ◆ 启动Hiveserver2
 - ◆ “netstat -ntlp|grep 10011”检查Hiveserver2是否已经启动，并且绑定到10011端口

```
cd /opt/linuxsir/hive
```

```
\\ hiveserver2默认端口为10000，这里改为10011
```

```
./bin/hive --service hiveserver2 --hiveconf hive.server2.thrift.port=10011&
```

```
\\ 等待hiveserver2启动
```

```
netstat -ntlp|grep 10011
```

Hive简介、部署与开发

- ◆ Hive安装、配置和运行
 - ◆ **Hiveserver2与beeline**
 - ◆ 如果beeline和hiveserver2在一个机器上；用如下方法，启动beeline

```
cd /opt/linuxsir/hive  
./bin/beeline
```

```
\\或者./bin/beeline -u jdbc:hive2://192.168.31.129:10011 -n root -p rootroot
```

- ◆ 在beeline提示符下执行如下命令

```
!connect jdbc:hive2://
```

```
\\ 用户名密码为root/rootroot
```

```
\\ 在beeline提示符下，执行SQL语句
```

```
select * from words;
```

```
select * from words order by id;
```

```
!quit
```

Hive简介、部署与开发

- ◆ Hive安装、配置和运行
 - ◆ Hiveserver2与beeline
 - ◆ 如果需要从另外一台机器启动beeline, 存取Hiveserver2

```
cd /opt/linuxsir/hive  
./bin/beeline
```

```
\\ 在beeline提示符下执行如下命令  
!connect jdbc:hive2://192.168.31.129:10011  
\\ 用户名密码为root/rootroot
```

```
\\ 在beeline提示符下, 执行SQL语句  
select * from words;  
select * from words order by id;  
select id, count(*) from words group by id order by id;  
!quit
```

Hive简介、部署与开发

- ◆ Hive安装、配置和运行
 - ◆ Hiveserver2与beeline
 - ◆ 停止metastore和Hiveserver2, 可以使用如下命令
 - ◆ 停止YARN和HDFS

```
\\停止metastore, 用如下命令  
netstat -ntlp |grep 19083  
kill -9 4688 \\4688是metastore进程号
```

```
\\停止hiveserver2, 用如下命令  
netstat -ntlp|grep 10011  
kill -9 11745 \\11745是hiveserver2进程号
```

```
cd /opt/linuxsir/hadoop/sbin  
./stop-yarn.sh  
./stop-dfs.sh
```

pause



Hive简介、部署与开发

- ◆ Hive安装、配置和运行
 - ◆ Hiveserver2与beeline
 - ◆ 问题1：提示“User: root is not allowed to impersonate root (state=,code=0)”。解决办法如下

编辑/opt/linuxsir/hadoop/etc/hadoop/core-site.xml，增加如下内容。

```
<!-- 用户“root”可以代理所有主机上的所有用户-->  
<property>  
  <name>hadoop.proxyuser.root.hosts</name>  
  <value>*</value>  
</property>  
<property>  
  <name>hadoop.proxyuser.root.groups</name>  
  <value>*</value>  
</property>
```

- ◆ 注意传播配置，在启动HDFS/YARN/metastore/Hiveserver2做实验

```
scp /opt/linuxsir/hadoop/etc/hadoop/core-site.xml root@192.168.31.130:/opt/linuxsir/hadoop/etc/hadoop  
scp /opt/linuxsir/hadoop/etc/hadoop/core-site.xml root@192.168.31.131:/opt/linuxsir/hadoop/etc/hadoop
```

Hive简介、部署与开发

- ◆ Hive安装、配置和运行
 - ◆ 其他问题1: hive启动出错

将 hive-site.xml 中的`${system:java.io.tmpdir}`, 替换成`/opt/linuxsir/hive/tmp`。
将 hive-site.xml 中的`${system:user.name}`, 替换成`${user.name}`

Hive简介、部署与开发

- ◆ Hive安装、配置和运行
 - ◆ 其他问题2: hive jdbc client的安全认证出问题

修改/opt/linuxsir/hive/conf/hive-site.xml, 具体如下

```
<property>
  <name>hive.server2.authentication</name>
  <value>NONE</value>
</property>

<property>
  <name>hive.server2.thrift.client.user</name>
  <value>root</value>
  <description>Username to use against thrift client</description>
</property>
<property>
  <name>hive.server2.thrift.client.password</name>
  <value>rootroot</value>
  <description>Password to use against thrift client</description>
</property>
```

Hive简介、部署与开发

◆ Hive安装、配置和运行

◆ 问题处置

- ◆ 需要注意的是，每次更新配置文件，需要按照如下步骤关闭软件、复制配置文件、以及重新启动软件，进行实验。
- ◆ (1) 停止服务进程，比如停止Hiveserver2，停止metastore，停止YARN，停止HDFS。
- ◆ (2) 在hd-master节点上修改配置文件，把配置文件复制到hd-slave1和hd-slave2节点。HDFS和YARN是多点部署的，所以需要进行配置文件的传播；而hive是单点部署的，所以不需要这个操作。
- ◆ (3) 重新启动服务进程，比如启动hdfs，启动yarn，启动metastore，启动hiveserver2，再做实验，比如通过beeline连接hiveserver2进行查询。
- ◆ 读者不难发现，停止服务的顺序和启动服务的顺序正好相反，符合进程间的依赖关系

pause



Hive简介、部署与开发

- ◆ Hive安装、配置和运行
- ◆ HWI是Hive Web Interface的缩写，是Hive的Web管理界面
- ◆ HWI的配置请参考文献<http://hive.praveendeshmane.co.in/hive/hive-web-interface-hwi.jsp>
- ◆ 启动HWI服务的命令如下.

```
cd /opt/linuxsir/hive  
bin/hive -service hwi &
```

Hive简介、部署与开发

- ◆ Hive安装、配置和运行
- ◆ Hive部署总结

Hive 层↵	(1) 用户通过 beeline 运行 HQL, beeline 连接 HiveServer2, HiveServer2 通过 metastore 组件查询 MySQL 获得元信息, HiveServer 调用 YARN 执行 MapReduce Job; ↵ (2) 用户也可以直接使用 Hive CLI 命令行工具运行 HQL, Hive CLI 命令行工具通过 metastore 组件查询 MySQL 获得元信息, 并且调用 YARN 执行 MapReduce Job。↵	↵	↵	↵
YARN 层↵	ResourceManager↵	NodeManager↵	NodeManager↵	...↵
HDFS 层↵	NameNode↵ Secondary NameNode↵	DataNode↵	DataNode↵	...↵
Hardware 各个节点↵	hd-master 节点↵ 192.168.31.129↵	hd-slave1 节点↵ 192.168.31.130↵	hd-slave2 节点↵ 192.168.31.131↵	...↵

Hive 和 Hadoop 的进程及其调用关系↵

pause



Hive简介、部署与开发

- ◆ Hive安装、配置和运行
- ◆ **Hive的Java开发**
- ◆ 在Java程序里存取Hive数据库，和编写普通的JDBC程序是一样的
- ◆ 完整的Java代码如下。这个实例首先建立到Hiveserver2(即Hive Thrift Server)的连接，然后执行SQL，显示查询结果

```
package com.pai.mvndemo;  
  
import java.sql.Connection;  
import java.sql.DriverManager;  
import java.sql.ResultSet;  
import java.sql.SQLException;  
import java.sql.Statement;
```

Hive简介、部署与开发

- ◆ Hive安装、配置和运行
- ◆ Hive的Java开发

```
public class App {  
    public static void main(String[] args) throws SQLException {  
        Connection conn;  
        Statement stmt;  
        ResultSet res;  
        String sql;  
  
        try {  
            String driverName = "org.apache.hive.jdbc.HiveDriver";  
            Class.forName(driverName);  
            Connection con = null;  
            // 端口号默认为10011，根据实际情况修改;  
            // 用户名: root, 密码: rootroot  
            con = DriverManager.getConnection("jdbc:hive2://192.168.31.129:10011/default", "root", "rootroot");  
            stmt = con.createStatement();  
            res = null;  
  
            sql = "SELECT * FROM words";  
  
            System.out.println("Running" + sql);  
            res = stmt.executeQuery(sql);  
            while (res.next()) {  
                System.out.println("id: " + res.getInt(1) + "\tword: " + res.getString(2));  
            }  
  
        } catch (Exception e) {  
            e.printStackTrace();  
            System.out.println("error");  
        }  
    }  
}
```

Hive简介、部署与开发

- ◆ Hive安装、配置和运行
- ◆ Hive的Java开发
- ◆ 如果用Maven进行Java编程，利用pom.xml文件，很方便进行项目依赖的设置，无需用户为项目导入jar包。pom.xml文件的具体内容如下

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-
4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.pai</groupId>
  <artifactId>mvndemo</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <packaging>jar</packaging>

  <name>mvndemo</name>
  <url>http://maven.apache.org</url>

  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>
```

Hive简介、部署与开发

- ◆ Hive安装、配置和运行
- ◆ Hive的Java开发
- ◆ 如果用Maven进行Java编程，利用pom.xml文件，很方便进行项目依赖的设置，无需用户为项目导入jar包。pom.xml文件的具体内容如下

```
<dependencies>
  <dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>3.8.1</version>
    <scope>test</scope>
  </dependency>

  <dependency>
    <groupId>org.apache.hive</groupId>
    <artifactId>hive-jdbc</artifactId>
    <version>1.2.1</version>
  </dependency>

  <dependency>
    <groupId>org.apache.hadoop</groupId>
    <artifactId>hadoop-common</artifactId>
    <version>2.7.3</version>
  </dependency>

  <dependency>
    <groupId>jdk.tools</groupId>
    <artifactId>jdk.tools</artifactId>
    <version>1.8</version>
    <scope>system</scope>
    <systemPath>C:/Program Files/Java/jdk1.8.0_131/lib/tools.jar</systemPath>
  </dependency>
</dependencies>

</project>
```

Hive简介、部署与开发

- ◆ Hive安装、配置和运行
- ◆ Hive的Java开发
 - ◆ 关于如何安装Maven，给Eclipse安装和配置Maven插件，以及新建Maven项目，请参考教材8.4节
 - ◆ 为了运行和调试该代码，应该在集群上启动HDFS、YARN、Hive metastore、以及Hiveserver2。结束以后，按照逆序停止Hiveserver2、Hive metastore、YARN、HDFS

pause



Hive简介、部署与开发

◆ Tez查询引擎

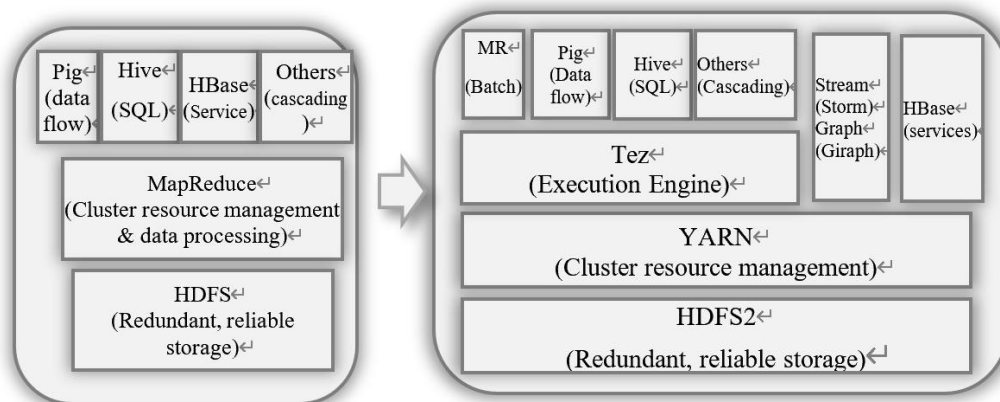
- ◆ Hive on Mapreduce把SQL翻译成Mapreduce作业(Job)运行, 效率较低
- ◆ 主要的原因, 是每个迭代(Iteration)都作为一个MapReduce作业实现, 这个作业从HDFS读取数据, 并且把中间结果写入HDFS
- ◆ 这种执行模式, 造成很大的延迟
- ◆ 我们可以把执行引擎改为tez, 提高Hive查询性能

Hive简介、部署与开发

- ◆ Tez查询引擎
- ◆ Tez是Apache构建在Hadoop之上的支持分布式DAG (Directed Acyclic Graph) 的计算框架
 - ◆ (1) 支持通过复杂的DAG描述并处理大数据任务，将多个有依赖的作业转换为一个作业，提高作业执行效率
 - ◆ (2) Cost-Based Optimization, 选择合适的执行计划
 - ◆ (3) 配合列存储格式，向量化查询执行模式，提高查询执行效率

Hive简介、部署与开发

- ◆ Hive安装、配置和运行
- ◆ Tez查询引擎
- ◆ 为了解决Hadoop/MapReduce计算模型执行过程中的延迟问题，HortonWorks提出了Tez，作为Apache培育项目(Incubator Project)。Tez项目的目标，是建立一个执行框架，支持大数据上的以DAG(Directed Acyclic Graph)表达的作业的处理。它是Hadoop2.0平台上可扩展的、高效的执行引擎。Hive、Pig或者Cascading作业，通过Tez引擎来运行，可以运行得更快。
- ◆ Hadoop2.0的提出，使得在Hadoop平台上执行用DAG表达的作业成为可能



Hadoop 1.0 仅仅支持 1 种计算模型，Hadoop2.0 支持多种计算模型。

Hive简介、部署与开发

- ◆ Hive安装、配置和运行
- ◆ Tez查询引擎
- ◆ 把数据处理逻辑建模成一个DAG连接起来的任务
 - ◆ 有向无环图DAG(Directed Acyclic Graph), 定义一个数据处理应用程序的数据处理流程。其中, DAG的顶点表示数据处理任务, 它反映了一定的业务逻辑(Business Logic), 即如何对数据进行转换和分析。DAG的边, 表示数据在不同顶点间的传递。
 - ◆ Tez把每个顶点建模成Input、Processor、和Output模块的组合。Input和Output模块决定了数据的格式, 以及数据应该从哪里读, 写入到哪里去等。Input代表一个顶点的输入来源(或者管道)。通过Input, Processor可以从HDFS、或者另外一个DAG顶点的输出, 获得输入数据。Output则代表一个顶点的输出的去向(管道)。通过Output, 一个Processor可以为另外一个DAG顶点递送数据, 或者把生成的数据保存到HDFS中。Processor则包装了数据转换和处理的逻辑, 它从一个或者多个Input消费数据, 进行相关处理, 然后产生多个Output。用户把不同的Input、Processor、Output模块组合成顶点。在此之上, 创建DAG数据处理工作流, 执行特定数据处理逻辑

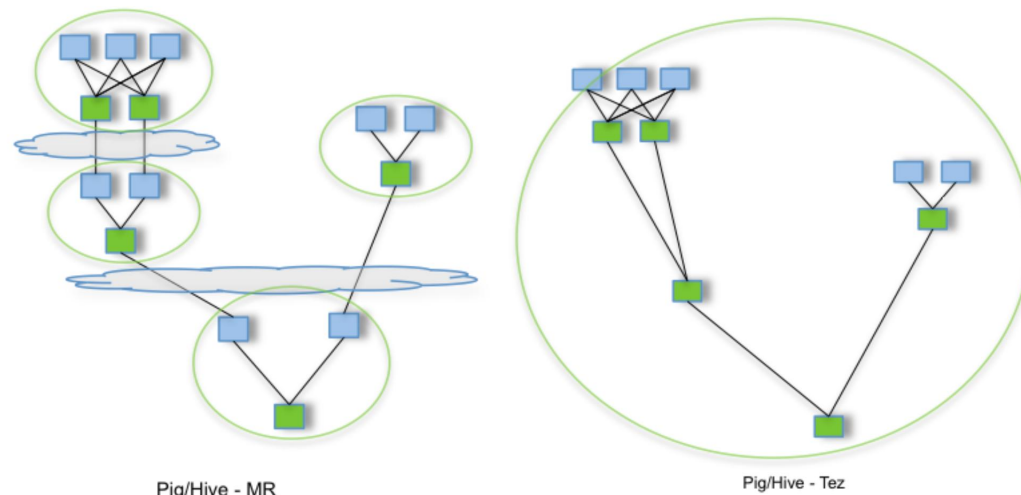
Hive简介、部署与开发

- ◆ Hive安装、配置和运行
- ◆ Tez查询引擎
- ◆ 把数据处理逻辑建模成一个DAG连接起来的任务

The 2 main design themes for Tez are:

- Empowering end users by:
 - Expressive dataflow definition APIs
 - Flexible Input-Processor-Output runtime model
 - Data type agnostic
 - Simplifying deployment
- Execution Performance
 - Performance gains over Map Reduce
 - Optimal resource management
 - Plan reconfiguration at runtime
 - Dynamic physical data flow decisions

By allowing projects like Apache Hive and Apache Pig to run a complex DAG of tasks, Tez can be used to process data, that e



Hive简介、部署与开发

- ◆ Hive安装、配置和运行
- ◆ Tez查询引擎
- ◆ 把数据处理逻辑建模成一个DAG连接起来的任务
 - ◆ 关于Tez的安装、配置和运行, 请读者参考官方文档
 - ◆ 目前, Spark SQL(另一个SQL on Hadoop系统)在性能上优于Tez, 并且社区活跃, 形成了对Tez的替代



Apache TEZ® / Apache TEZ® 0.9.2

GETTING STARTED

[Overview](#)
[Talks and Meetup Recordings](#)
[Project License](#)

DOCUMENTATION

[Install Guide](#)
[Local Mode](#)
[Tez UI](#)
[Tez Shuffle Handler](#)

Apache TEZ® 0.9.2

- [Download Release Artifacts](#)
- [Release Notes](#)
- [Documentation](#)
 - [API Javadocs](#) : Documentation for the Tez APIs
 - [Runtime Library Javadocs](#) : Documentation for built-in implement
 - [Tez Mapreduce Javadocs](#) : Documentation for built-in implement
 - [Tez Configuration](#) : Documentation for configurations of Tez. The
 - [Tez Runtime Configuration](#) : Documentation for runtime configu



Lightning-fast unified analytics engine

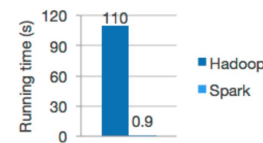
[Download](#) [Libraries](#) [Documentation](#) [Examples](#) [Community](#) [Developers](#)

Apache Spark™ is a unified analytics engine for large-scale data processing.

Speed

Run workloads 100x faster.

Apache Spark achieves high performance for both batch and streaming data, using a state-of-the-art DAG scheduler, a query optimizer, and a physical execution engine.



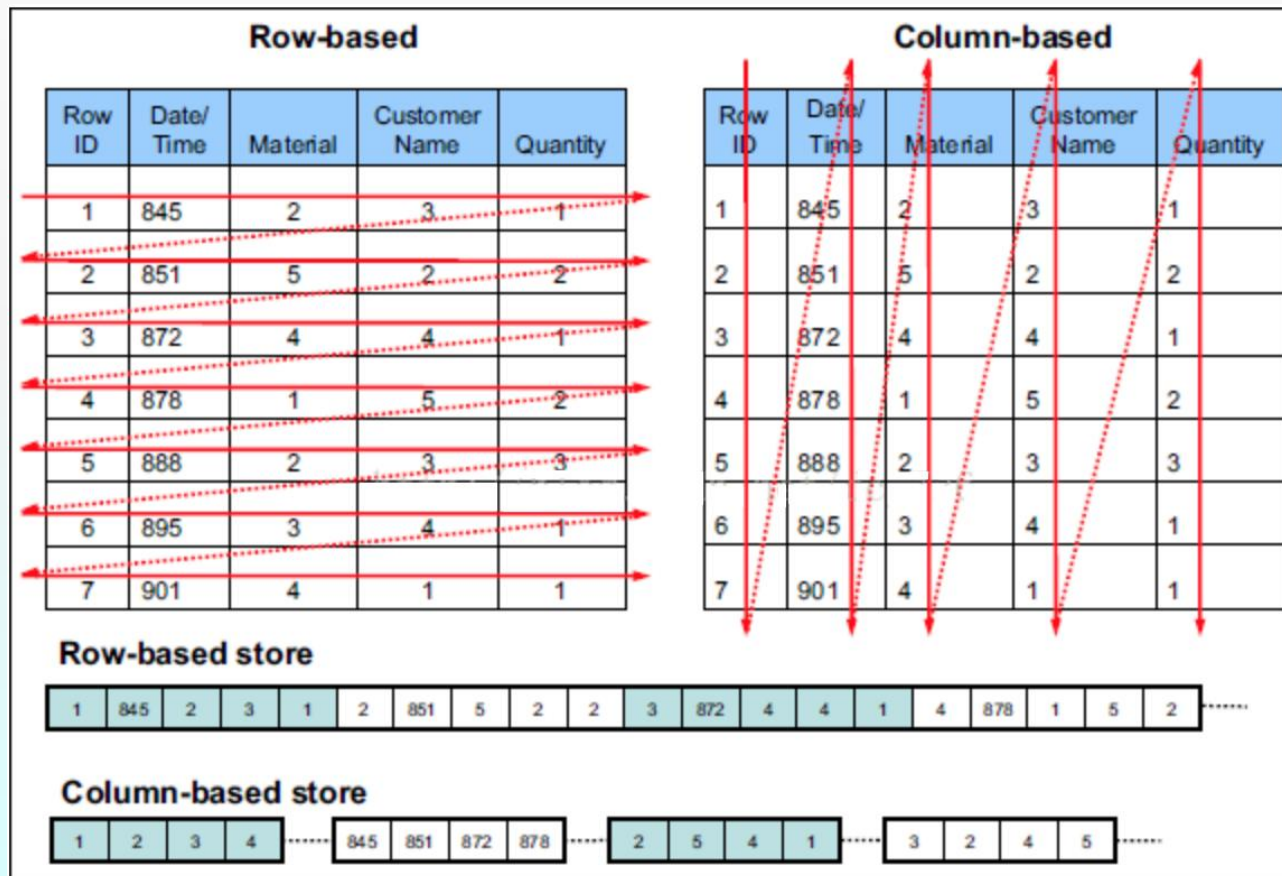
Logistic regression in Hadoop and Spark

pause



Hive

◇ 行存储、列存储及其优缺点



Hive

◆ 行存储、列存储及其优缺点

◆ 优点

- ◆ 更新和插入比较方便，写入效率高
- ◆ 容易实现事务性、一致性控制

◆ 缺点

- ◆ 对于分析型查询，如果只查某几个字段，有可能存取不必要的数据

◆ 应用

- ◆ 事务型应用

Hive

◆ 行存储、列存储及其优缺点

◆ 优点：吞吐量高，性能强

- ◆ 适合压缩
- ◆ 提高I/O效率、提高CPU Cache效率
- ◆ 针对某几个字段的读取效率较高

◆ 缺点

- ◆ 一致性、事务性较弱

◆ 应用

- ◆ 分析型应用

Hive简介、部署与开发

- ◆ **Hadoop平台上的列存储技术及其优势**
- ◆ 在分析型应用中，列存储技术具有若干明显的优势：
 - ◆ (1) 更少的I/O操作：读取数据的时候，可以把投影下推(Project Pushdown)，只需读取查询需要的列，可以大大减少每次查询的I/O数据量。甚至可以支持谓词下推(Predicate Pushdown)，跳过不满足条件的数据行。通过减少不必要的数据扫描，提高查询性能。
 - ◆ (2) 更高的压缩比：每一列中的数据，类型相同，由于相同类型的数据连续存放，可以使用针对性的编码和压缩方法对数据进行压缩，大大降低数据存储空间。
 - ◆ (3) 由于每一列的数据类型相同，可以使用更加适合CPU流水线的编码方式，减少CPU的缓存失效(Cache Miss)
 - ◆ **Hadoop生态圈涌现出一系列列存储格式，包括RC File、ORC、Parquet等；这里介绍Parquet格式**

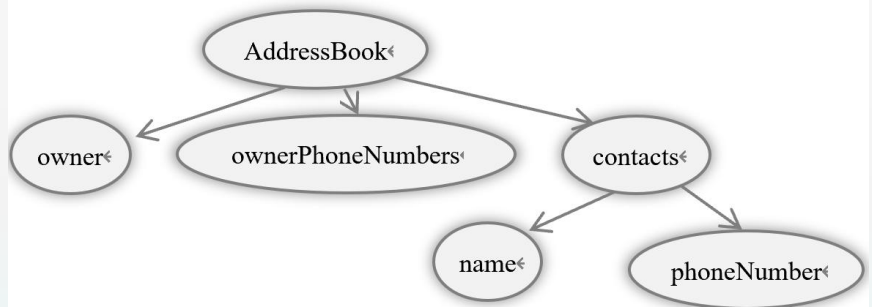
Hive简介、部署与开发

- ◆ Hadoop平台上的列存储技术及其优势
- ◆ **Parquet格式**
 - ◆ Parquet是Hadoop平台上的一种列存储格式。Parquet的灵感来自于2010年Google发表的关于Dremel系统的论文。该论文介绍了一种支持嵌套结构的列存储格式，以提升查询性能。Parquet为Hadoop生态系统中的所有项目，提供支持高压缩率的列存储格式。它兼容各种数据处理框架、对象模型，支持各种查询引擎(Hive、Impala、Presto等)，与编程语言无关。
 - ◆ Parquet最初由Twitter和Cloudera(Impala的开发者)合作开发完成，然后开源
 - ◆ 2015年5月，Parquet从Apache的孵化器里孵化完成，成为Apache顶级项目

Hive简介、部署与开发

- ◆ Hadoop平台上的列存储技术及其优势
- ◆ **Parquet数据模型**

```
message AddressBook {  
  required string owner;  
  repeated string ownerPhoneNumbers;  
  repeated group contacts {  
    required string name;  
    optional string phoneNumber;  
  }  
}
```



AddressBook 对应的 Schema

Hive简介、部署与开发

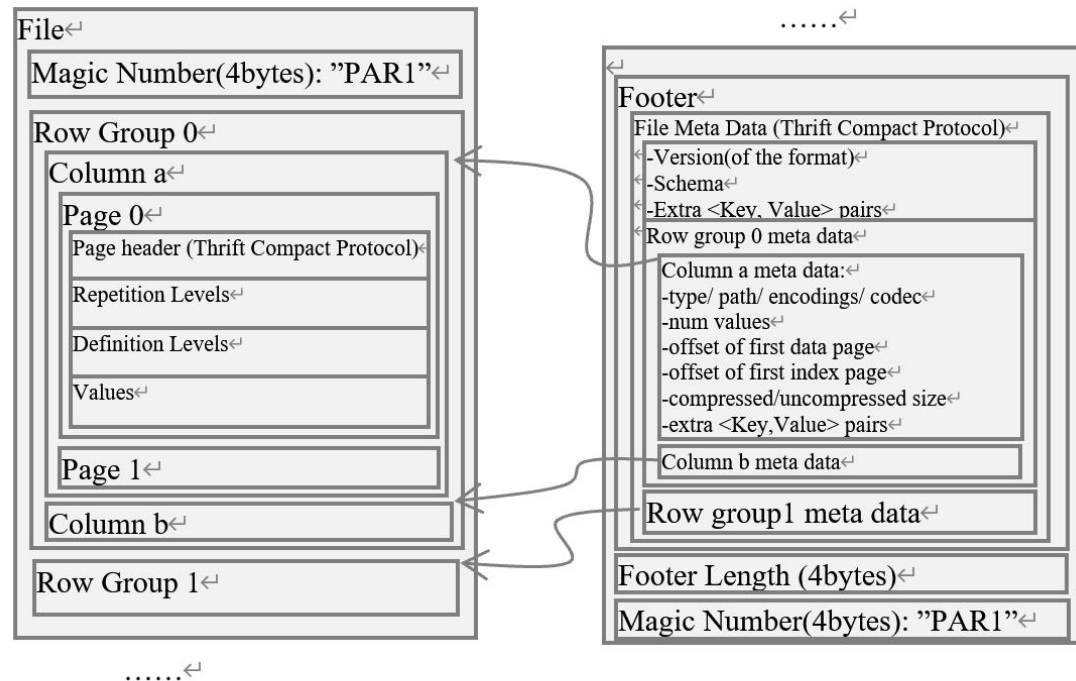
- ◆ Hadoop平台上的列存储技术及其优势
- ◆ **Parquet数据模型**
- ◆ 如何把内存中的每个AddressBook对象，按照列存储格式保存到硬盘文件中？在Parquet格式的存储中，一个数据模式(Schema)的树结构有几个叶子节点，实际的存储中就会有多少列(Column)。于是，上述实例中的AddressBook数据模式，在存储上共有四个列(Column)

Column↵	Type↵	↵	AddressBook↵			
owner↵	string↵	↵	owner↵	ownerPhoneNumbers↵	contacts↵	
ownerPhoneNymbers↵	string↵	↵			name↵	phoneNumber↵
contacts.name↵	string↵	↵	...↵	...↵	...↵	...↵
contacts.phoneNumber↵	string↵	↵	...↵	...↵	...↵	...↵
		↵	...↵	...↵	...↵	...↵

AddressBook 实际存储的列↵

Hive简介、部署与开发

- ◆ Hadoop平台上的列存储技术及其优势
- ◆ **Parquet文件结构**
- ◆ Parquet文件结构涉及如下几个重要的层次，如图所示。这是一种自描述的文件结构



Parquet 文件结构

Hive简介、部署与开发

◆ Parquet文件结构

- ◆ (1) HDFS文件(File): 一个HDFS文件, 包括数据和元数据。数据存储在多个Block中。
- ◆ (2) HDFS数据块(Block): 它是HDFS上最小的存储单位。HDFS把一个Block保存到本地磁盘, 并且在不同的机器上, 维护另外两个副本。通常情况下, 一个Block的大小为256M、512M等
- ◆ (3) 行组(Row Group): 按照行, 将数据(表格)物理上划分为多个组。每一个行组, 包含一定的行数。一个行组包含该行组数据的各个列对应的列块。一般建议采用更大的行组大小(512MB-1GB)。更大的行组, 意味着更大的列块, 有利于在硬盘上做串行I/O
- ◆ 由于一次可能需要读取整个行组, 所以一般让一个行组刚好在一个HDFS数据块中, HDFS数据块的大小需要设得更大。比如, 采用1GB的HDFS数据块大小, 1个HDFS数据块放一个行组
- ◆ (4) 列块(Column Chunk): 在一个行组中, 每一列保存在一个列块中。行组中的所有列连续地存储在这个行组中。不同的列块, 可以使用不同的算法进行压缩。一个列块由多个页组成
- ◆ (5) 页(Page): 每一个列块, 划分为多个页, 页是压缩和编码的单元。在同一个列块内的不同页, 可以使用不同的编码方式。典型的页面大小为1MB(After Encoding)

Hive简介、部署与开发

◆ Parquet文件结构

- ◆ 列存储给数据压缩提供了更大的发挥空间。除了我们常见的Snappy、GZIP等通用压缩方法以外，由于同一列的数据类型是一致的，可以使用不同的压缩算法

不同压缩算法及其应用场景

压缩算法	使用场景
行程编码(Run Length Encoding)	重复数据
Delta 编码(Delta Encoding)	有序数据集，例如 Timestamp，自动生成的 ID，以及监控的各种指标(Metric)
字典编码(Dictionary Encoding)	小规模的数据集合，例如 IP 地址
前缀编码(Prefix Encoding)	字符串的 Delta Encoding

Hive简介、部署与开发

◆ Parquet文件结构

- ◆ 在数据库系统中，最常用的查询优化手段之一是谓词下推
- ◆ 通过将一些过滤条件尽可能早地执行，可以减少查询计划后续需要处理的数据量，从而提升性能

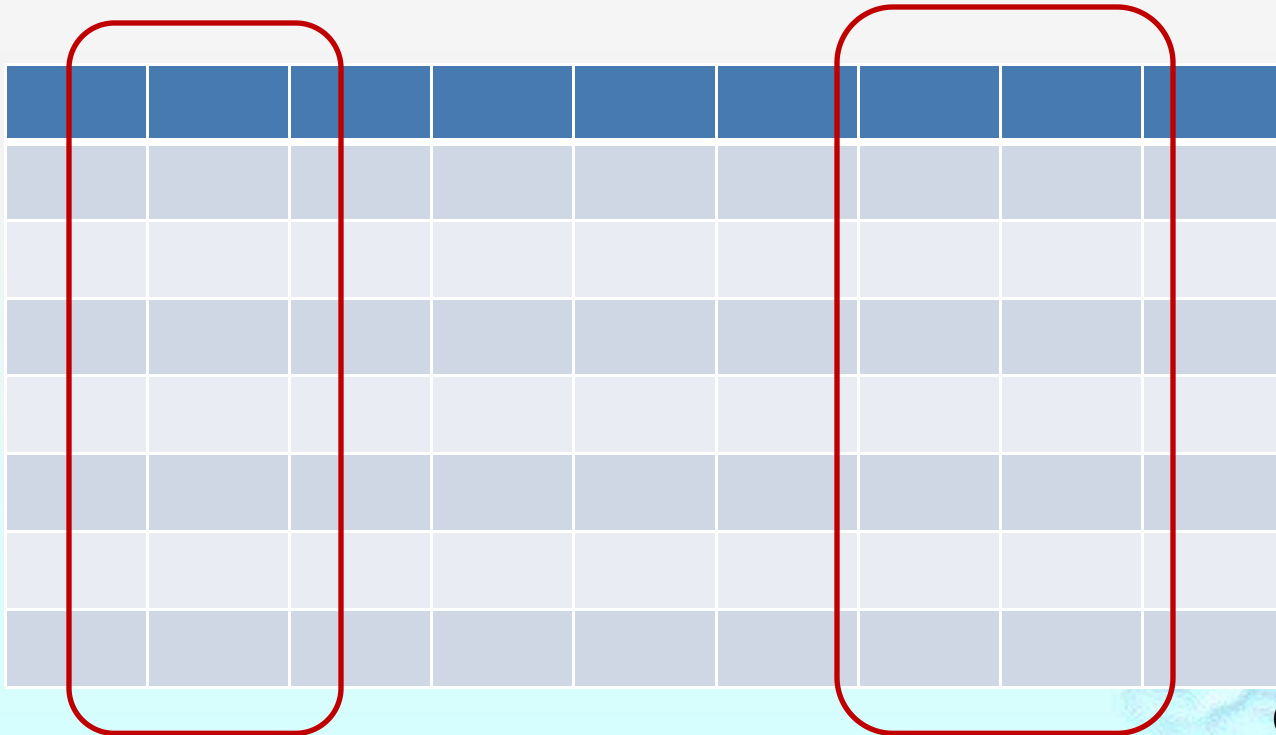
```
Select Count(*)  
From A Join B On A.id = B.id  
Where A.a >10 And B.b < 100
```

- ◆ 如果首先对A表和B表执行Table Scan操作，然后进行Join操作，再执行过滤，最后计算聚集函数返回，效率是不高的。
- ◆ 但是，如果把过滤条件A.a > 10和B.b < 100，分别移动到A表的Table Scan和B表的Table Scan的时候执行，遴选出符合条件的记录，就可以大大降低Join操作的输入数据，加快Join操作，并且提高整个查询的效率

Hive简介、部署与开发

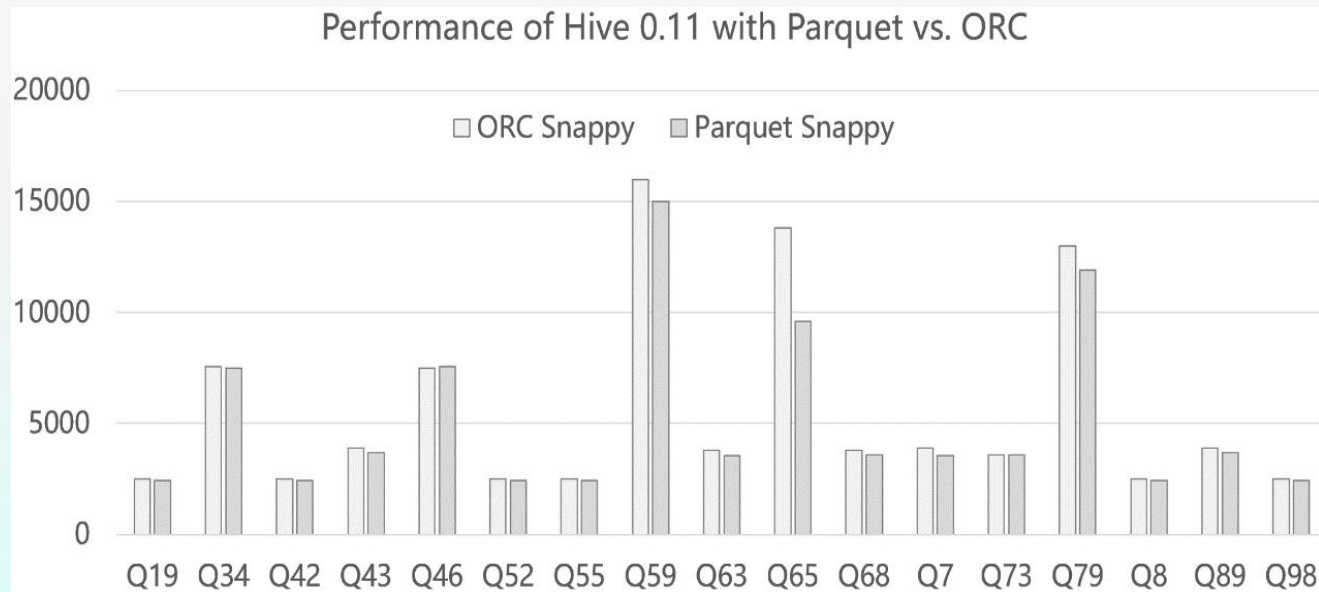
◆ Parquet文件结构

- ◆ 列存储格式的另外一个优势，是**映射下推**
- ◆ 在获取表中的数据的时候，只需要扫描查询中需要的列。由于避免了不必要的数据列的提取，查询的效率得到提高



Hive简介、部署与开发

- ◆ **Parquet的性能**
- ◆ Criteo公司在Hive数据仓库中，使用ORC和Parquet两种列存储格式，执行TPC-DS测试基准的结果



Hive简介、部署与开发

- ◆ **Parquet的性能**
- ◆ 在这个实验中，TPC-DS数据集的Scale Factor为100。相对于文本文件，ORC Snappy把文件压缩到原来文件大小的35%，Parquet Snappy把文件压缩到原来文件大小的33%
- ◆ 从测试结果可以看出，在数据存储方面，两种存储格式在使用Snappy压缩的情况下，占用的空间相差并不大，在查询性能上Parquet格式稍微好于ORC格式
- ◆ Parquet格式和ORC格式，各有优缺点
 - ◆ Parquet格式原生地支持嵌套式数据结构，而ORC对此支持得较差。Parquet不支持数据的修改和ACID事务语义，但是ORC对此提供支持
 - ◆ 注意，在OLAP环境下，我们很少会对单条数据进行修改，更多的是对数据进行批量导入

Hive简介、部署与开发

◆ Parquet的使用

- ◆ 如何在Hive里面使用Parquet列存储文件，可以参考文献
- ◆ <http://blog.cloudera.com/blog/2014/03/how-to-use-parquet-with-impala-hive-pig-mapreduce/>
- ◆ 在Spark SQL系统的实验部分，将介绍如何使用Parquet列存储格式
- ◆ <https://spark.apache.org/docs/latest/sql-data-sources-parquet.html>
- ◆ <https://kontext.tech/column/spark/257/write-and-read-parquet-files-in-hdfs-through-sparkscala>

pause

